

PyTorch 从零实战学习手册

从张量到手写数字识别 · 小白陪练笔记

行者 · 2026-08-16

配套仓库：[torch_demon](#) (uv 管理)

目录

- 1. 如何使用这本手册
- 2. 环境搭建 (uv + 镜像源 + Mac MPS)
- 3. 阶段1：张量 Tensor
- 4. 阶段2：自动求导 autograd
- 5. 阶段3：线性回归实战 (房价预测)
- 6. 阶段4：工业级模板 (nn.Module + DataLoader)
- 7. 阶段5：分类任务 Classification
- 8. 阶段5b：加深对照 (加层 / 参数量)
- 9. 阶段6：MNIST 手写数字分类
- 10. 应用：手写数字识别器
- 11. 踩坑合集 (环境 / 数据 / 代码)
- 12. 下一步：CNN 路线图
- 附录：命令速查表

■■■■■■

```
# 1)  X  y 
```

```
X = (X_raw - X_mean) / X_std
y = (y_data - y_mean) / y_std

model = torch.nn.Linear(3, 1)
opt = torch.optim.Adam(model.parameters(), lr=0.01)

for step in range(2000):
    pred = model(X)
    loss = ((pred - y) ** 2).mean() # MSE
    opt.zero_grad(); loss.backward(); opt.step()

# 2) 
```

```
w_true_scale = model.weight.detach() * (y_std / X_std)
b_true_scale = y_mean - (w_true_scale * X_mean).sum() + model.bias.detach()
#  0.49/2.07/-1.53/10.9 0.5/2.0/-1.5/10
```

图1：房价回归收敛过程与学出的规律

学出的系数接近真实规律，并能解释「为什么标签 y 也要标准化」。

把阶段3的手写写法，重写成 PyTorch

```
class HouseModel(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.net = torch.nn.Linear(3, 1)
    def forward(self, x):
        return self.net(x)

model = HouseModel()
loader = DataLoader(TensorDataset(X, y), batch_size=16, shuffle=True)
opt = torch.optim.Adam(model.parameters(), lr=0.01)
loss_fn = torch.nn.MSELoss()

model.train()
for epoch in range(50):
    for xb, yb in loader:
        loss = loss_fn(model(xb), yb)
        opt.zero_grad(); loss.backward(); opt.step()
```


过关标准

能说清 CrossEntropyLoss 与 MSELoss 的区别、输出维度为何等于类别数、如何用 argmax + 准确率评估。

8. 阶段5b：加深对照（加层 / 参数量）

把网络从 1 个隐藏层加到 3 个（2→64→32→16→3），对比基线。加深能拟合更复杂的边界。

- 加层规则：nn.Sequential 中每层 Linear(in,out) 的 in/out 维度必须接力；
- 输出层不加 ReLU（要保留原始 logits 给 CrossEntropyLoss）；
- 参数量手算：第一层 $2 \times 64 + 64 = 192$ ，第二层 $64 \times 32 + 32 = 2080$ ，第三层 $32 \times 16 + 16 = 528$ ，第四层 $16 \times 3 + 3 = 51$ ，合计 2851；
- 不是越深越好：过深会过拟合、梯度消失，玩具数据上 100% 准确率两者都能到，但真实任务要权衡。

过关标准

能动手加隐藏层、保证维度链 in/out 接力、手算参数量（2→64→32→16→3 = 2851）。

9. 阶段6：MNIST 手写数字分类

终于上真实数据集：MNIST（6 万张 28×28 手写数字）。模型是一个 MLP。

- 图片是 $28 \times 28 = 784$ 个像素，必须先 Flatten 拉平成 784 维向量，才能进 Linear；
- Flatten 只是「reshaping」，不改变数值，只是把空间结构拍平（这也是 MLP 的短板来源）；
- 模型与数据必须在同一 device（mps / cpu）；
- 预处理：ToTensor([0,1]) + Normalize(0.1307, 0.3081)；
- 结果：测试集准确率约 97.35%（纯 MLP 的教科书区间 95%~98%）。

```
class MNISTNet(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = torch.nn.Flatten()           # (B,1,28,28) -> (B,784)
        self.net = torch.nn.Sequential(
            torch.nn.Linear(784, 128),
            torch.nn.ReLU(),
            torch.nn.Linear(128, 10),               # 10 ■
        )
    def forward(self, x):
        return self.net(self.flatten(x))
```

为什么不是 100%？因为 Flatten

丢掉了空间结构（不知道哪两个像素相邻），且对位置/形变不鲁棒。这正是下一阶段 CNN 要解决的——CNN 在 MNIST 上轻松到 99%+。

过关标准

能说清图片为何要先 Flatten 才能进 Linear、模型与数据需同 device、测试集准确率约 95%~98%。

10. 应用：手写数字识别器

把刚训好的模型接上一个鼠标画板（gradio Sketchpad），写个数字实时识别，右侧输出概率条形 + 0~9 概率明细表。

- 复用 MNISTNet(784→128→ReLU→10)；
- 首次运行自动训练 5~10 epoch 并缓存 mnist_mlp.pth，之后秒开；

- 预处理铁律：推理必须与训练完全一致（灰度 → 取笔画 → 缩放居中 28×28 → `Normalize(0.1307, 0.3081)`），否则识别失真；
- 取笔画：把所有图层的「墨迹」（与各自背景差异大的像素）取并集，再渲染成黑底白字——这样白底黑笔 / 黑底白笔都能正确识别。

```
def preprocess(img):
    # gradio Sketchpad dict: {"image": , "mask": }
    layers = [img[k] for k in ("mask", "image") if k in img] or list(img.values())
    grays = [np.array(l, float).mean(2) if np.array(l).ndim==3 else np.array(l, float)
             for l in layers]
    ink = np.zeros_like(grays[0], bool)
    for g in grays:
        bg = np.median(g)
        ink |= np.abs(g - bg) > 25
    digit = (ink * 255).astype("uint8")
    #  ->  28x28 -> ToTensor -> Normalize
    ...
    return (t - 0.1307) / 0.3081
```

运行：`uv add gradio && uv run handwriting_app.py`，打开终端打印的 <http://127.0.0.1:7860>。

两个曾踩的坑：① Sketchpad 回调传的是 dict 不是 PIL，直接 `.convert()` 会报错；② 若按「白像素最多的图层」选笔画，会选中纯白背景，导致恒输出同一个数字（如恒为 5）。已用「墨迹并集」修复。

11. 踩坑合集（环境 / 数据 / 代码）

环境

- `uv add` 装大依赖超时 → 配 `~/config/uv/uv.toml` 顶层 `index-url` 清华镜像（不要写 `[install]` 表，也不要只设环境变量）；
- Mac M4 走 MPS（自带支持）；AMD 显卡 ROCm 坑多，学习阶段用 CPU 即可。

数据

- 标签要用原始特征生成、归一化只作用于输入 X；
- 输入 X 和标签 y 都要标准化，否则 Adam 步长太小不收敛；
- 梯度会累加，每个 batch 前必须 `zero_grad()`；
- MNIST 下载源不稳定：torchvision 对 `.gz` 做 MD5 校验，不同镜像重新压缩后 MD5 会变 → 死循环。正确做法是用 `download_mnist.py` 拉官方 `.gz` → 校验 → 解压出 `.ubyte` → 校验内容 → 落盘到 `data_mnist/MNIST/raw/`。

代码

- `CrossEntropyLoss` 内部含 softmax，模型输出别再 softmax；
- 加层时维度链 in/out 必须接力，输出层不加 ReLU；
- 模型和数据必须同一 device；
- 应用：Sketchpad 返回 dict、且不能按白像素选笔画层（见第 10 章）。

12. 下一步：CNN 路线图

MLP 把图片拍扁、丢了空间结构。卷积网络（CNN）用一个小窗口在图上滑动提取局部特征（权值共享），天生对位置/形变鲁棒，MNIST 上能到 99%+。

```
class CNN(torch.nn.Module):
    def __init__(self):
```



```
super().__init__()
self.net = torch.nn.Sequential(
    torch.nn.Conv2d(1, 16, 3), torch.nn.ReLU(), torch.nn.MaxPool2d(2),
    torch.nn.Conv2d(16, 32, 3), torch.nn.ReLU(), torch.nn.MaxPool2d(2),
    torch.nn.Flatten(),
    torch.nn.Linear(32*7*7, 128), torch.nn.ReLU(),
    torch.nn.Linear(128, 10),
)
def forward(self, x):
    return self.net(x)
```

- Conv2d(1→16, kernel=3)：用 3×3 窗口提取 16 种局部特征（边缘/角/笔画）；
- MaxPool2d(2)：把图缩小一半，降维且让特征对位置不敏感；
- 两次卷积+池化后特征图变为 32×7×7，展平接入全连接；
- 核心一句话：卷积=局部特征提取（权值共享）；池化=缩小+抗平移。

学完 CNN，就可以把应用 handwriting_app.py 的底层换成 CNN，识别率从 97% 冲到 99%+。

附录：命令速查表

目的	命令
复原环境	uv sync
逐课运行	uv run lessons/01_tensor.py
房价预测	uv run main.py
工业模板	uv run lessons/04_template.py
分类/加深	uv run lessons/05_classification.py / 05b_deeper.py
MNIST	uv run lessons/06_mnist.py（源失效先跑 download_mnist.py）
手写识别应用	uv add gradio && uv run handwriting_app.py
生成本手册	uv run --extra "" python build_learning_pdf.py

—— 手册完。回到代码里多跑几遍，比反复读更重要。